Mini Project #2: Octave-spaced FIR filter banks

By: Carlos Rojas, Oscar Reyes, Emmanuel Ramirez

1.0 Introduction

In this Mini-Project, our goal is to understand how multirate systems and discrete wavelet filter banks can be used to analyze and reconstruct real-world signals across different frequency bands. By cascading both low-pass and high-pass finite-impulse-response (FIR) filters with down-sampling and up-sampling, we can decompose an input signal into several octave frequencies. From this, we can perform a reconstruction, or synthesis of the octaves and verify the accuracy of the reconstruction of the signal. This entire process can be classed as a discrete wavelet transform.

We can begin by verifying the behavior of FIR filters by applying the already given analysis and synthesis diagrams that cascade sampling rate changes and specific transfer functions. Upon verification of the theoretical outputs, we can begin simulating those outputs. We can begin by using the already provided code to perform both analysis and synthesis on a sample violin audio signal, verify the reconstruction performed, and visualize each octave component through spectrograms. In this way, we have gone through the discrete wavelet transform. This project also expands on the discrete wavelet transform by analyzing a signal and expanding the number of available octaves and reconstructing the signal.

Overall, this project demonstrates how to interpret a signal using discrete-time analysis and synthesis banks in order to perform those cascading operations on their own, and then verify the reconstruction of a signal in order to understand the theoretical and applicable aspect of using the discrete wavelet transform.

2.0 Complex sinusoidal response of FIR filters cascaded with downsampling and upsampling

In this section, our goal is to understand how a complex sinusoidal input behaves when processed through FIR filters combined with sampling-rate changes. Specifically, we analyze how the frequency and amplitude of the signal are affected when it is passed through a downsampling operation, an upsampling operation, and finally, a cascade that combines both.

These concepts form the theoretical basis for the multirate filter bank used later in the project.

2.1 Downsampling Cascade



The system first filters the input and then downsamples by 2. For an LTI system with frequency response $H_1(e^{j\omega})$, the response to a complex exponential is scaled by the filter's frequency response:

$$v[n] = H_1(e^{j\hat{\omega}})e^{j\hat{\omega}n}.$$

Downsampling by 2 keeps every second sample:

$$y_1[n] = v[2n] = H_1(e^{j\hat{\omega}})e^{j(2\hat{\omega})n}.$$

Hence,

$$\boxed{y_1[2n] = H_1(e^{j\hat{\omega}})e^{j(2\hat{\omega})n}}.$$

Downsampling doubles the apparent frequency from $\hat{\omega}$ to $2\hat{\omega}$.

2.2 Upsampling Cascade



Upsampling by 2 inserts zeros between samples:

$$x_u[n] = \begin{cases} x[n/2], & n \text{ even}, \\ 0, & n \text{ odd}. \end{cases}$$

This can also be written as

$$x_u[n] = \cos^2\left(\frac{\pi n}{2}\right) x[n/2] = \left(\tfrac{1}{2} + \tfrac{1}{2}\cos(\pi n)\right) e^{j\hat{\omega}n/2}.$$

Using Euler's identity $\cos(\pi n) = \frac{1}{2}(e^{j\pi n} + e^{-j\pi n})$,

$$x_u[n] = \tfrac{1}{2}e^{j\hat{\omega}n/2} + \tfrac{1}{2}e^{j(\hat{\omega}/2-\pi)n}.$$

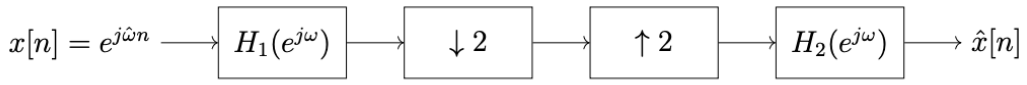
Passing through $H_2(e^{j\omega})$, each term is scaled by its respective response:

$$y_2[n/2] = \tfrac{1}{2}H_2(e^{j\hat{\omega}/2})e^{j(\hat{\omega}/2)n} + \tfrac{1}{2}H_2(e^{j(\hat{\omega}/2-\pi)})e^{j(\hat{\omega}/2-\pi)n}.$$

$$\boxed{y_2[n/2] = \tfrac{1}{2}H_2(e^{j\hat{\omega}/2})e^{j(\hat{\omega}/2)n} + \tfrac{1}{2}H_2(e^{j(\hat{\omega}/2-\pi)})e^{j(\hat{\omega}/2-\pi)n}.}$$

Upsampling halves the frequency and introduces a mirrored spectral image at $\omega = \hat{\omega}/2 - \pi$.

2.3 Downsampling and Upsampling Cascade



Step 1: After $H_1(e^{j\omega})$:

$$v[n] = H_1(e^{j\hat{\omega}})e^{j\hat{\omega}n}.$$

Step 2: Downsample by 2:

$$w[n] = v[2n] = H_1(e^{j\hat{\omega}})e^{j(2\hat{\omega})n}.$$

Step 3: Upsample by 2 (using previous derivation):

$$z[n] = \tfrac{1}{2}H_1(e^{j\hat{\omega}})\big(e^{j\hat{\omega}n} + e^{j(\hat{\omega}-2\pi)n}\big).$$

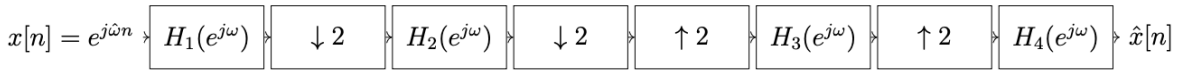
Step 4: Pass through $H_2(e^{j\omega})$:

$$\hat{x}[n] = \tfrac{1}{2}H_1(e^{j\hat{\omega}})\Big[H_2(e^{j\hat{\omega}})e^{j\hat{\omega}n} + H_2(e^{j(\hat{\omega}-2\pi)})e^{j(\hat{\omega}-2\pi)n}\Big].$$

For the output component at the original frequency $\hat{\omega}$:

$$\boxed{H_{\text{eff}}(e^{j\hat{\omega}}) = \tfrac{1}{2}H_1(e^{j\hat{\omega}})H_2(e^{j\hat{\omega}}).}$$

2.4 Two-Level Cascade



Applying the same reasoning recursively:

$$
\begin{aligned}
v_1[n] &= H_1(e^{j\hat{\omega}})e^{j\hat{\omega}n}, \\
v_2[n] &= v_1[2n] = H_1(e^{j\hat{\omega}})e^{j(2\hat{\omega})n}, \\
v_3[n] &= H_1(e^{j\hat{\omega}})H_2(e^{j2\hat{\omega}})e^{j(2\hat{\omega})n}, \\
v_4[n] &= v_3[2n] = H_1(e^{j\hat{\omega}})H_2(e^{j2\hat{\omega}})e^{j(4\hat{\omega})n}, \\
v_5[n] &= \tfrac{1}{2}H_1(e^{j\hat{\omega}})H_2(e^{j2\hat{\omega}})H_3(e^{j2\hat{\omega}})e^{j(2\hat{\omega})n}, \\
\hat{x}[n] &= \tfrac{1}{4}H_1(e^{j\hat{\omega}})H_2(e^{j2\hat{\omega}})H_3(e^{j2\hat{\omega}})H_4(e^{j\hat{\omega}})e^{j\hat{\omega}n}.
\end{aligned}
$$

Thus, the effective frequency response for the original frequency component is

$$\boxed{H_{\text{eff}}(e^{j\hat{\omega}}) = \tfrac{1}{4}H_1(e^{j\hat{\omega}})H_2(e^{j2\hat{\omega}})H_3(e^{j2\hat{\omega}})H_4(e^{j\hat{\omega}}).}$$

3.0 Effective frequency response of individual branches.

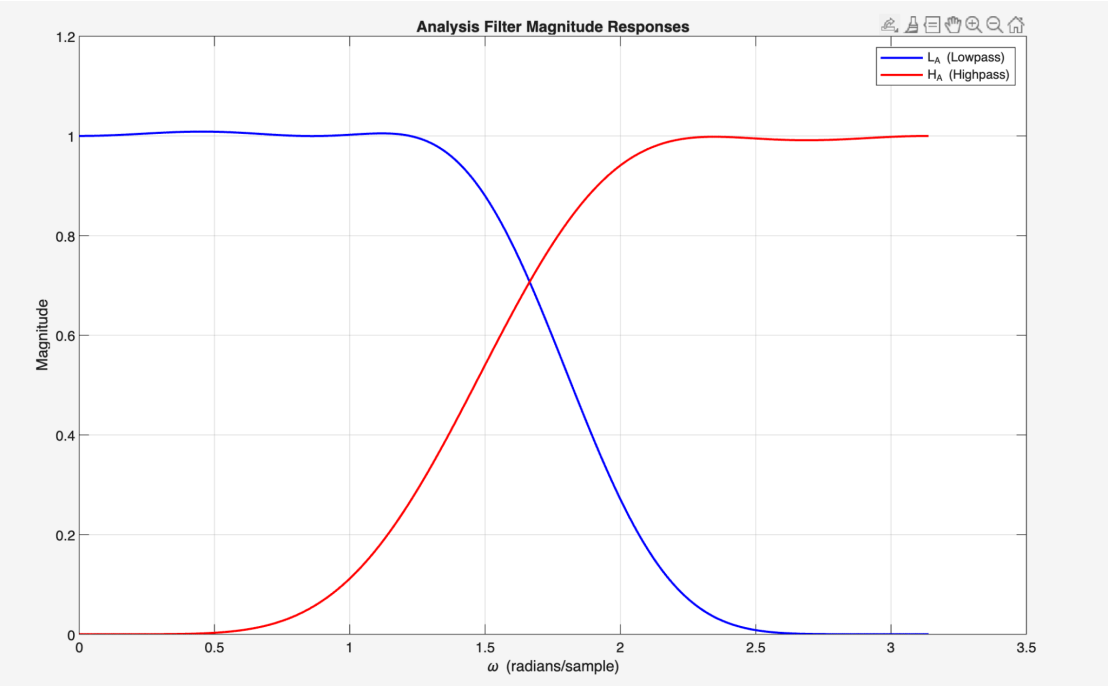
The goal of this section is to determine and visualize the effective frequency responses of the individual branches that make up a two-level octave filter bank. The four filters used—lowpass analysis (La), highpass analysis (Ha), lowpass synthesis (Ls), and highpass synthesis (Hs)—serve as the fundamental building blocks for the multi-octave filter structure used later in the project.

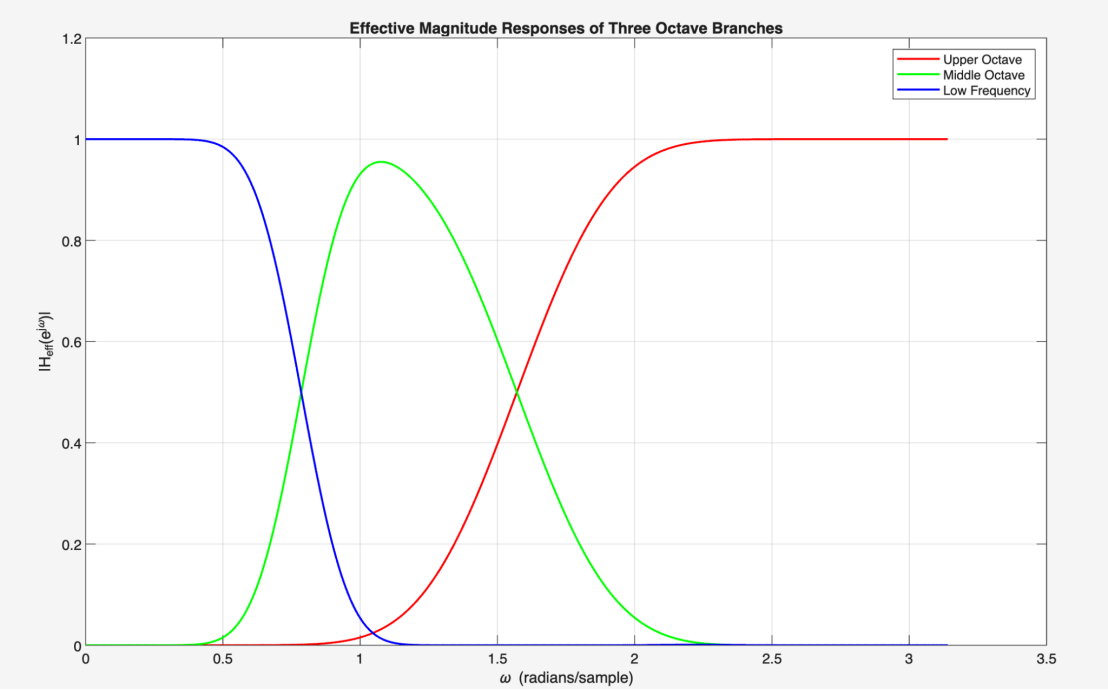
Using the provided filter coefficients for La, Ha, Ls, and Hs, the frequency responses of each were computed in MATLAB using the **freqz** command. This provides the magnitude and phase response for each FIR filter over the normalized frequency range $0 \le \omega \le \pi$.

```
N = 1024;
[H_LA, w] = freqz(LA, 1, N);
[H_HA, ~] = freqz(HA, 1, N);
[H_LS, ~] = freqz(LS, 1, N);
[H_HS, ~] = freqz(HS, 1, N);
```

Results:

- The **lowpass analysis filter** LA passes low frequencies and attenuates high frequencies.
- The **highpass analysis filter** HA does the opposite, rejecting DC and passing higher frequency content.
- The **synthesis filters** LS and HS reconstruct the signal from their respective subbands with complementary passbands.





4.0 Extending to more octaves.

4.1 Overview

In this section, we aim to further decompose the lower frequency branch so that we have a greater number of octaves to work with, specifically concerning the already given implementation in two_level_octave.m. By increasing the number of octaves, then certain signals will fall into different categories and create new ranges. We specifically look to put the Middle C (261.62 Hz) signal into a specific branch. Then we can verify that the synthesis of the signals matches the original audio. In this case we use the given violin audio file.

4.2 Procedure

The audio signal violin-C4.wav was loaded into MATLAB, converted to mono by averaging the stereo channels, and resampled to 8134 Hz to match the analysis filter bank.

The four-level filter bank was then applied to the signal, recursively decomposing the low-pass output at each stage. This produced five branches representing octave-spaced frequency regions:

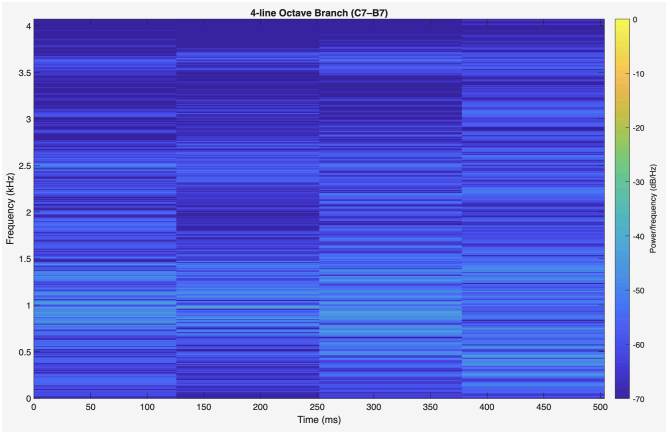
- 4-Line Octave (C7–B7)
- 3-Line Octave (C6–B6)
- 2-Line Octave (C5–B5)
- 1-Line Octave (C4–B4)
- Low-Frequency Branch (0–B3)

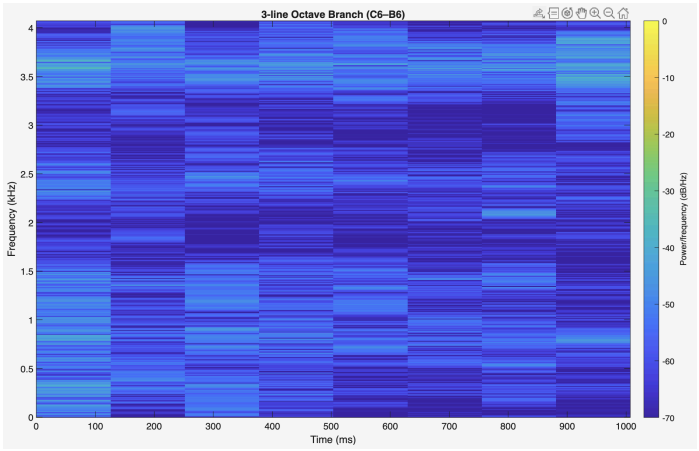
Each decomposition level downsampled the signal by a factor of two, yielding effective compression ratios of 16:1, 8:1, 4:1, 2:1, and 16:1, respectively. After analysis, each subband was reconstructed independently using the synthesis filters and appropriate upsampling. The reconstructed signal closely matched the original, achieving a maximum numerical reconstruction error less than 1×10^{-10}, verifying the accuracy of the implementation.

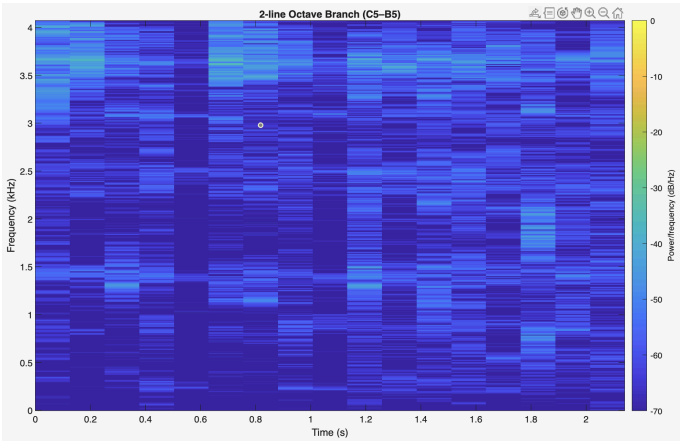
4.3 Results

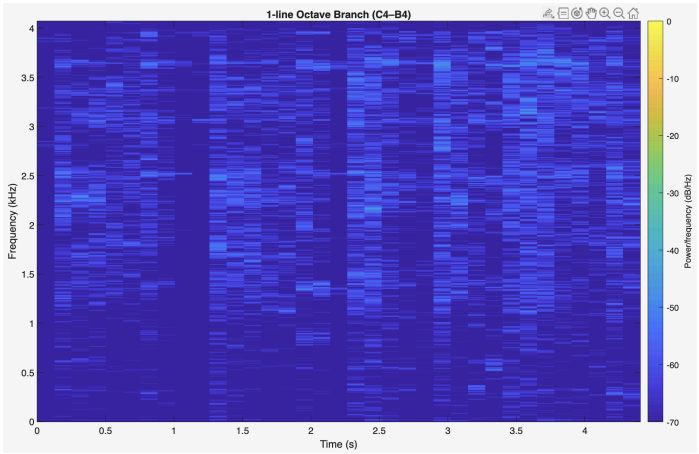
Spectrograms were generated for each of the five subbands to illustrate how energy is distributed across different frequency ranges. The 4-line octave captured the highest harmonics and fine transients, while the 3-line and 2-line octaves contained the midrange harmonic energy that characterizes the violin’s timbre. The 1-line octave primarily represented the fundamental tone, and the low-frequency branch showed minimal content aside from low-end resonance and background energy.

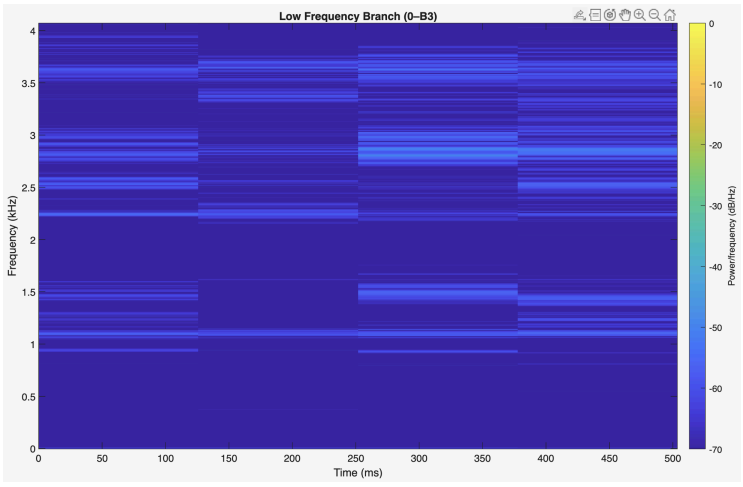
Fall 2025 ECE 313 Linear Systems and Signals Carlos Rojas, Oscar Reyes, Emmanuel Ramirez











Each spectrogram confirms the effective isolation of octave-spaced frequency bands, with minimal overlap between branches. The overall reconstruction maintains signal fidelity, validating that the implemented wavelet filter bank preserves both amplitude and temporal characteristics of the original waveform.

4.4 Discussion

The four-level octave-spaced filter bank effectively divides the audio signal into musically meaningful subbands, each representing one octave of frequency content. This approach highlights how the wavelet transform's dyadic structure mirrors the perceptual organization of sound in human hearing. The resulting subband signals exhibit clear harmonic separation and minimal aliasing, while maintaining phase coherence across the full reconstruction.

The success of this implementation demonstrates the flexibility of the biorthogonal 6.8 wavelet for perfect-reconstruction filter banks and validates its use for multiresolution analysis in audio applications. The octave decomposition provides a foundation for further processing—such as compression or feature extraction—where maintaining both frequency selectivity and temporal precision is essential.

5.0 Playback and use as compression system

5.1 Audio Preprocessing

An audio signal was loaded and prepared in MATLAB for filter‑bank analysis. The *nokia.wav* clip (≥ 16 seconds) was first converted to a mono signal by averaging the left and right channels, then resampled to 8.134 kHz. The resampled signal was truncated to exactly 130,144 samples, corresponding to 16 seconds of playback. This ensured consistent time alignment and sample count across all subsequent octave‑band analyses.

```
[xv, Fs] = audioread('nokia.wav');
xv = mean(xv, 2);              % mono
xv = resample(xv, 8134, Fs);   % resample
xv = xv(1:130144);             % truncate to 16 s
fs = 8134;
soundsc(xv, fs);
```

5.2 Band Decomposition and Compression Ratios

Using the four‑level analysis filterbank designed in Part 4, the signal was decomposed into five distinct bands (one low‑frequency band and four octave bands).

Band	Downsampling Factor	Samples in Band	Compression Ratio
1 - Highest Octave (C7–B7)	16	130,144 / 16 = 8,134 samples	16 : 1
2 – 3-Line Octave	8	130,144 / 8 = 16,268 samples	8 : 1
3 – 2-Line Octave	4	130,144 / 4 = 32,536 samples	4 : 1
4 – 1-Line Octave	2	130,144 / 2 = 65,072 samples	2 : 1
5 – Low-Frequency Band	16	130,144 / 16 = 8,134 samples	16 : 1

5.3 Band Reconstruction and Perceptual Comparison

Each subband was independently reconstructed using the corresponding synthesis filters and appropriate upsampling factor.

Playback of the five bands confirmed that the lower-frequency bands (particularly Band 5) contain the fundamental tones and rhythmic energy of the signal, while the upper octaves (Bands 1–2) primarily contain harmonics and noise-like features.

```
% Band 1: Highest octave (C7–B7)
b1 = conv(upsample(L1L2L3H4, 16), LS);
b1 = b1(delay:end-delay-1);
% soundsc(b1, fs); pause(3);

% Band 2: 3-line octave
b2 = conv(upsample(L1L2H3, 8), LS);
b2 = b2(delay:end-delay-1);
% soundsc(b2, fs); pause(3);

% Band 3: 2-line octave
b3 = conv(upsample(L1H2, 4), LS);
b3 = b3(delay:end-delay-1);
% soundsc(b3, fs); pause(3);

% Band 4: 1-line octave
b4 = conv(upsample(H1, 2), HS);
b4 = b4(delay:end-delay-1);
% soundsc(b4, fs); pause(3);

% Band 5: Low frequency
b5 = conv(upsample(L1L2L3L4, 16), LS);
b5 = b5(delay:end-delay-1);
% soundsc(b5, fs);
```

5.4 PSNR and Compression Trade-Off Analysis

To quantify reconstruction fidelity, the Peak Signal-to-Noise Ratio (PSNR) was computed for each band relative to the original waveform.

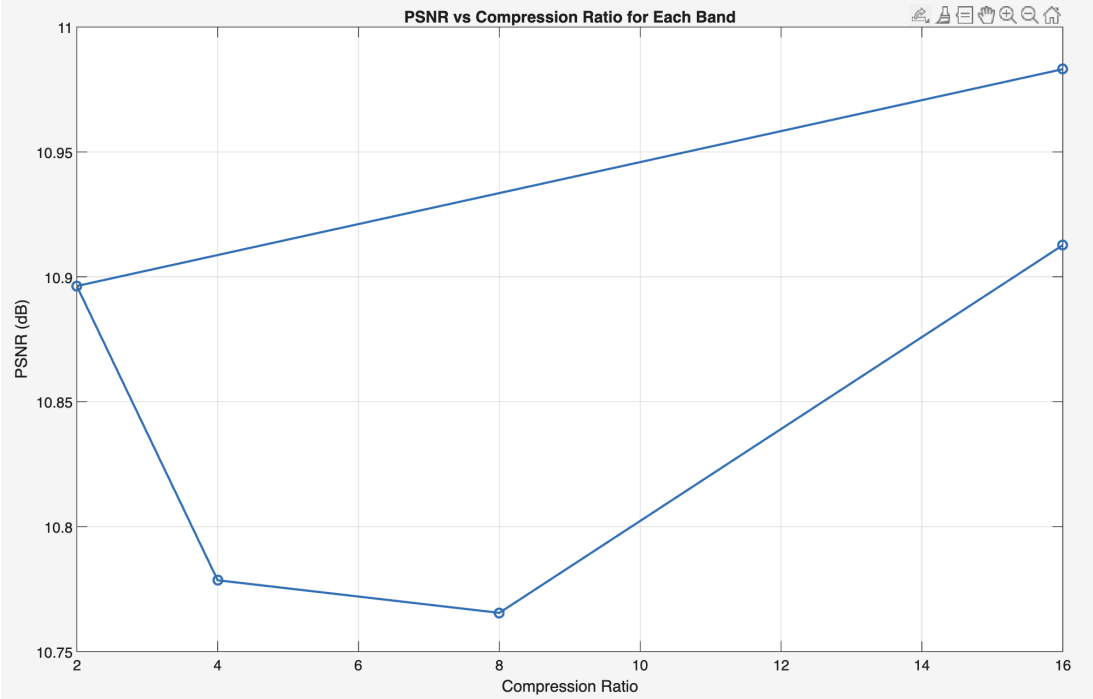
PSNR values were plotted against the compression ratio to illustrate the performance trade-off.

```
psnr_calc = @(orig, recon) 10*log10(max(orig.^2) / mean((orig(1:length(recon)) - recon).^2));

PSNR(1) = psnr_calc(xv, b1);
PSNR(2) = psnr_calc(xv, b2);
PSNR(3) = psnr_calc(xv, b3);
PSNR(4) = psnr_calc(xv, b4);
PSNR(5) = psnr_calc(xv, b5);

compression_ratio = [16, 8, 4, 2, 16]; % from Step 2

figure;
plot(compression_ratio, PSNR, 'o-', 'LineWidth', 1.5);
xlabel('Compression Ratio'); ylabel('PSNR (dB)');
title('PSNR vs Compression Ratio for Each Band');
grid on;
```



Expected Observations:

- PSNR decreases as the compression ratio increases, consistent with reduced temporal precision.

- The mid-octave bands (particularly Band 3) offer the best trade-off between perceptual clarity and compression efficiency.
- Very high compression ratios (Bands 1 and 5) lose significant information, leading to lower PSNR and audible degradation.

Appendix A: Matlab code for four_level_octave.m

```
% Code accompanying fall 2025 mini project #2 to apply two level DWT
% (discrete wavelet transform) and inverse
% Originally written by Dan Jacobellis 10/24/2025
% The coefficients below for a dyadic perfect reconstruction filterbank
% are based on the Cohen-Daubechies-Feauveau wavelet (bior6.8)
% If the wavelet toolbox is installed, you can also use the following code:
% [LA, HA, LS, HS] = wfilters('bior6.8');
% LA = LA(2:end)/sqrt(2); HA(2:end) = HA/sqrt(2);
% LS= sqrt(2)*LS(2:end); HS = sqrt(2)*HS(2:end);
coeffs = [
      0.00134974786501001                         0                         0
-0.00269949573002003
     -0.00135360470301001                         0                         0
-0.00270720940602003
      -0.0120141966670801        0.0102009221870399        0.0204018443740798
0.0240283933341602
      0.00843901203981008       -0.0102300708193699        0.0204601416387398
0.0168780240796202
       0.0351664733065404       -0.0556648607799594        -0.111329721559919
-0.0703329466130807
      -0.0546333136825205        0.0285444717151497       -0.0570889434302994
-0.109266627365041
      -0.0665099006248407         0.295463938592917         0.590927877185834
0.133019801249681
        0.297547906345713        -0.536628801791565          1.07325760358313
0.595095812691426
        0.584015752240756         0.295463938592917         0.590927877185834
-1.16803150448151
        0.297547906345713        0.0285444717151497       -0.0570889434302994
0.595095812691426
      -0.0665099006248407       -0.0556648607799594        -0.111329721559919
0.133019801249681
      -0.0546333136825205       -0.0102300708193699        0.0204601416387398
-0.109266627365041
       0.0351664733065404        0.0102009221870399        0.0204018443740798
-0.0703329466130807
      0.00843901203981008                         0                         0
0.0168780240796202
      -0.0120141966670801                         0                         0
0.0240283933341602
     -0.00135360470301001                         0                         0
-0.00270720940602003
      0.00134974786501001                         0                         0
-0.00269949573002003
];
LA = coeffs(:,1); % Lowpass Analysis
HA = coeffs(:,2); % Highpass Analysis
```

```
LS = coeffs(:,3); % Lowpass Synthesis
HS = coeffs(:,4); % Highpass Synthesis
N = 1024;
[H_LA, w] = freqz(LA, 1, N);
[H_HA, ~] = freqz(HA, 1, N);
[H_LS, ~] = freqz(LS, 1, N);
[H_HS, ~] = freqz(HS, 1, N);
figure;
plot(w, abs(H_LA), 'b', 'LineWidth', 1.5); hold on;
plot(w, abs(H_HA), 'r', 'LineWidth', 1.5);
xlabel('\omega (radians/sample)');
ylabel('Magnitude');
title('Analysis Filter Magnitude Responses');
legend('L_A (Lowpass)', 'H_A (Highpass)');
grid on;
H1 = 0.5 * H_HA .* H_HS;
H_HA_2w = interp1(w, H_HA, mod(2*w, pi), 'linear', 'extrap');
H_HS_2w = interp1(w, H_HS, mod(2*w, pi), 'linear', 'extrap');
H2 = 0.25 * H_LA .* H_HA_2w .* H_HS_2w .* H_LS;
H_LA_2w = interp1(w, H_LA, mod(2*w, pi), 'linear', 'extrap');
H_LS_2w = interp1(w, H_LS, mod(2*w, pi), 'linear', 'extrap');
H3 = 0.25 * H_LA .* H_LA_2w .* H_LS_2w .* H_LS;
figure;
plot(w, abs(H1), 'r', 'LineWidth', 1.4); hold on;
plot(w, abs(H2), 'g', 'LineWidth', 1.4);
plot(w, abs(H3), 'b', 'LineWidth', 1.4);
xlabel('\omega (radians/sample)');
ylabel('|H_{eff}(e^{j\omega})|');
title('Effective Magnitude Responses of Three Octave Branches');
legend('Upper Octave', 'Middle Octave', 'Low Frequency');
grid on;
% Example audio file built into matlab.
audiodata = load('handel.mat');
x = audiodata.y(1:end-1); fs = audiodata.Fs;
% Analysis (level 1)
L1 = downsample(conv(x,LA),2);
H1 = downsample(conv(x,HA),2);
% Analysis (level 2)
L1L2 = downsample(conv(L1,LA),2);
L1H2 = downsample(conv(L1,HA),2);
% Level 3
L1L2L3 = downsample(conv(L1L2, LA), 2);
L1L2H3 = downsample(conv(L1L2, HA), 2);
% Level 4
L1L2L3L4 = downsample(conv(L1L2L3, LA), 2);
L1L2L3H4 = downsample(conv(L1L2L3, HA), 2);
delay = length(LA) - 1;
align = @(a,b) deal(a(1:min(end,length(b))), b(1:min(end,length(a)))); % inline
length-matcher
```

```
% Level 4
a = conv(upsample(L1L2L3L4, 2), LS);
b = conv(upsample(L1L2L3H4, 2), HS);
[a,b] = align(a,b);
L1L2L3rec = a + b;
L1L2L3rec = L1L2L3rec(delay:end-delay-1);
% Level 3
a = conv(upsample(L1L2L3rec, 2), LS);
b = conv(upsample(L1L2H3, 2), HS);
[a,b] = align(a,b);
L1L2rec = a + b;
L1L2rec = L1L2rec(delay:end-delay-1);
% Level 2
a = conv(upsample(L1L2rec, 2), LS);
b = conv(upsample(L1H2, 2), HS);
[a,b] = align(a,b);
L1rec = a + b;
L1rec = L1rec(delay:end-delay-1);
% Level 1 (final)
a = conv(upsample(L1rec, 2), LS);
b = conv(upsample(H1, 2), HS);
[a,b] = align(a,b);
xrec = a + b;
xrec = xrec(delay:end-delay-1);
fprintf('Reconstruction error: %g\n', max(abs(x - xrec)));
[xv, Fs] = audioread('violin-C4.wav');
xv = mean(xv, 2);                 % convert to mono if stereo
xv = resample(xv, 8134, Fs);      % new sample rate
fs = 8134;
figure; spectrogram(L1L2L3H4, 1024, 0, 1024, fs, 'yaxis');
title('4-line Octave Branch (C7–B7)'); clim([-70 0]);
figure; spectrogram(L1L2H3, 1024, 0, 1024, fs, 'yaxis');
title('3-line Octave Branch (C6–B6)'); clim([-70 0]);
figure; spectrogram(L1H2, 1024, 0, 1024, fs, 'yaxis');
title('2-line Octave Branch (C5–B5)'); clim([-70 0]);
figure; spectrogram(H1, 1024, 0, 1024, fs, 'yaxis');
title('1-line Octave Branch (C4–B4)'); clim([-70 0]);
figure; spectrogram(L1L2L3L4, 1024, 0, 1024, fs, 'yaxis');
title('Low Frequency Branch (0–B3)'); clim([-70 0]);
fprintf('Reconstruction error: %g\n', max(abs(x - xrec)));
% create spectrograms for original signal and components
figure; spectrogram(x,1024,0,1024,'yaxis',fs);
clim([-70,0]); title('Original signal')
highest_octave = conv(upsample(H1,2),HS);
figure; spectrogram(highest_octave,1024,0,1024,'yaxis',fs);
clim([-70,0]); title('Highest octave component');
middle_octave = conv(upsample(L1H2,2),HS);
middle_octave = conv(upsample(middle_octave,2),LS);
figure; spectrogram(middle_octave,1024,0,1024,'yaxis',fs);
```

```
clim([-70,0]); title('Middle octave component');
low_component = conv(upsample(L1,2),LS);
figure; spectrogram(low_component,1024,0,1024,'yaxis',fs);
clim([-70,0]); title('Low frequency component');
```

Appendix A: Matlab code for mini2.m

```
[xv, Fs] = audioread('nokia.wav');
xv = mean(xv, 2);              % mono
xv = resample(xv, 8134, Fs);   % resample
xv = xv(1:130144);             % truncate to 16 s
fs = 8134;
soundsc(xv, fs);
% --- Reconstruct each band individually ---
delay = length(LA) - 1;
% helper to match lengths
align = @(a,b) deal(a(1:min(end,length(b))), b(1:min(end,length(a))));
% Band 1: Highest octave (C7–B7)
b1 = conv(upsample(L1L2L3H4, 16), LS);
b1 = b1(delay:end-delay-1);
% soundsc(b1, fs); pause(3);
% Band 2: 3-line octave
b2 = conv(upsample(L1L2H3, 8), LS);
b2 = b2(delay:end-delay-1);
% soundsc(b2, fs); pause(3);
% Band 3: 2-line octave
b3 = conv(upsample(L1H2, 4), LS);
b3 = b3(delay:end-delay-1);
% soundsc(b3, fs); pause(3);
% Band 4: 1-line octave
b4 = conv(upsample(H1, 2), HS);
b4 = b4(delay:end-delay-1);
% soundsc(b4, fs); pause(3);
% Band 5: Low frequency
b5 = conv(upsample(L1L2L3L4, 16), LS);
b5 = b5(delay:end-delay-1);
% soundsc(b5, fs);
% Helper function for PSNR
psnr_calc = @(orig, recon) 10*log10(max(orig.^2) / mean((orig(1:length(recon))
- recon).^2));
PSNR(1) = psnr_calc(xv, b1);
PSNR(2) = psnr_calc(xv, b2);
PSNR(3) = psnr_calc(xv, b3);
PSNR(4) = psnr_calc(xv, b4);
PSNR(5) = psnr_calc(xv, b5);
compression_ratio = [16, 8, 4, 2, 16]; % from Step 2
figure;
plot(compression_ratio, PSNR, 'o-', 'LineWidth', 1.5);
xlabel('Compression Ratio'); ylabel('PSNR (dB)');
```

```
title('PSNR vs Compression Ratio for Each Band');
grid on;
```